EX11 - Taylor with Functions - Multiple Return Values

Beginner Step-by-Step Guide

This guide follows the real file `ex11_taylor_exp_function.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Convert EX07 into a taylor_exp(x,N,xp) function that returns both numeric value and symbolic expression.

Problem variables / elements

Name	Meaning
x	symbol
N	order
xp	evaluation point

Equation or logic on paper

```
e^x ~ sum_{n=0..N} x**n/n!
```

Manual check

The mathematics is the same as EX07; the new idea is calling the same function with different N and xp values.

2. What we already know and what is new

Already seen:

- functions
- return
- Taylor
- lists

New in this exercise:

- returning two values
- unpacking return values
- mixing positional/keyword arguments
- reuse for convergence studies

3. New Python concepts

1. Multiple return

Syntax: `return ex_numeric, expression_to_show`

The function returns two values. Python allows them to be received into two separate names.

2. Unpacking

Syntax: `approximation, expression = taylor_exp(...)`

The first returned value maps to the first name and the second to the second.

3. Mixed arguments

Syntax: `taylor_exp(x, N=5, xp=0.5)`

Positional arguments must appear before keyword arguments.

4. Code step by step

Step 1. Define function

```
def taylor_exp(x, N, xp):
```

The function receives everything needed to build and evaluate the approximation.

Step 2. Validate N

```
if not isinstance(N, int) or N < 0:
    return None, None
```

Because the caller expects two values, the invalid case also returns two values.

Step 3. Return two results

```
return ex_numeric, expression_to_show
```

One is for numeric calculation and the other for displaying the polynomial.

Step 4. Receive both

```
approximation, expression_to_show = taylor_exp(x, N, xp)
```

This line executes the function and unpacks its two returns.

Step 5. Call several cases

```
taylor_exp(x, N=0, xp=0.5)
taylor_exp(x, N=5, xp=0.5)
taylor_exp(x, N=10, xp=2)
```

The function is reused to study approximation changes without duplicating the algorithm.

5. Execution flow

```
def taylor_exp
 -> validate N
 -> build polynomial
 -> return 2 values
 -> unpack
 -> compare cases
```

6. Expected result / check

For N=10 and xp=2, the approximation is ~7.388994709 versus exp(2)~7.389056099. N=-1 or N=3.5 return None,None.

7. Common beginner mistakes

- Expecting one return when there are two.
- Placing a positional argument after a keyword.
- Forgetting to return two values in the error path.
- Duplicating the algorithm for each case.

8. If you remember only five things

- A function can return multiple values.
- Returns unpack by position.
- Positional arguments come before keywords.
- Function reuse makes comparisons easier.
- The algorithm lives in one place.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.